

Data Batching in LLM Instruction Fine-tuning!

⑥ Step 2: Organizing data into training batches

Stage 1:
preparing
dataset

1) Dataset
downloading
and formatting

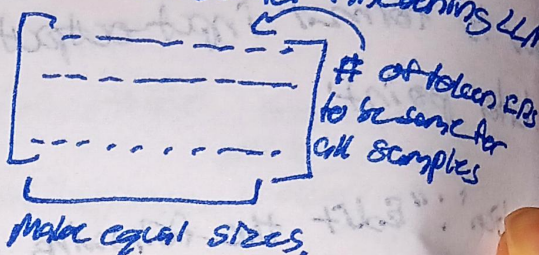
2) Batching the
dataset

3) Creating
Data loaders

We will learn
how to efficiently
pad them to equal
lengths, so we can
assemble multiple
instructions examples
in a batch.

Then we create
the PyTorch dataloaders
we will use for fine-tuning LLM

Sample 1
Sample 2
Sample 3



2.1) Format data
using prompt template

Below is an instruction that describes a task.
Write a response that appropriately
completes the request.

Instruction: ...
Response: ...

Format input into
an instruction-response
template

2.2) Tokenize
formatted data

[2106, 318, 281, 12064, 326,
8472, 257, 13, ...]

convert instruction-
response entry into
token IDs

2.3) Adjust to the
same length w/
padding tokens

(var a)
[2106, 818, 281, 12084, 326,
8472, 257, 13, ..., 50256,
50256]

Add end-of-text
tokens (50256) to pad
data samples to same
length

2.4) Create target
token IDs for
training

(var a)
[818, 281, 12064, 326, 8477,
257, 482, 13, ..., 50256,
50256, 50256, 50256]

create a list of
target IDs for the
model to learn (these
are the inputs shifted by 1,
plus an additional padding
token)

2.5) Replace padding
tokens w/
placeholder

(var a)
[818, 281, 12064, 326, 8477,
257, 482, 13, ..., 50256, 3,
-100, -100, -100]

Replace certain padding
tokens by -100 to
exclude them from the
training loss

CNN APP FOR BATH PLUGS

[2.3] ^{tokens}
 First batch $\left[\begin{array}{l} \text{Input 1 } [0, 1, 2, 3, 4] \rightarrow [0, 1, 2, 3, 4] \\ \text{Input 2 } [5, 6] \text{ len 2} \rightarrow [5, 6, 50256, 50256, 50256] \\ \text{Input 3 } [7, 8, 9] \text{ len 3} \rightarrow [7, 8, 9, 50256, 50256] \end{array} \right.$
 Add all training examples into batch so that they have same length to max length sample

GPT-2 $\rightarrow$ 50257 (vocab size) in range $[0, 50256]$ (end-of-text token (50257))
 Do same thing with subsequent batches (USB LOCAL BATCH LOAN TEST TOKEN LENGTH AS THE MAX LENGTH SAMPLE)
 second batch $\left[\begin{array}{l} \text{Input 4 } [8, 1] \rightarrow [8, 1, 50256, 50256] \\ \text{Input 5 } [10, 3, 11, 8] \rightarrow [10, 3, 11, 8] \\ \text{Input 6 } [5, 22, 13, 13] \rightarrow [5, 22, 13, 13] \end{array} \right.$
 Paddling token

[2.4] Target vector does not contain the first input ID
 Input 1 $[0, 1, 2, 3, 4]$
 Target 1 $[1, 2, 3, 4, 50256]$ Add end-of-text token (padding)

Tokens IDs in the target are similar to input IDs but shifted by 1 position

Input 2 $[5, 6, 50256, 50256, 50256]$
 Target 2 $[6, 50256, 50256, 50256, 50256]$

WE ARE NOW GOING NEXT TOKEN PREDICTIONS! TASKS!

Always add an end-of-text token to the target

[2.5] Target 1 $[1, 2, 3, 4, 50256] \rightarrow [1, 2, 3, 4, 50256]$
 Target 2 $[6, 50256, 50256, 50256, 50256] \rightarrow [6, 50256, 50256, 50256]$
 Target 3 $[8, 9, 50256, 50256, 50256] \rightarrow [8, 9, 50256, 50256]$

We don't modify the first instance of the end-of-text token

We replace all but the first instance of the end-of-text (padding) tokens w/ -100

Ignore Index = -100 FOR

CROSS ENTROPY LOSS

$\hookrightarrow$ Makes training more efficient

$\hookrightarrow$ Does not unnecessarily include unimportant tokens

Why should I learn everything? All I should

learn is the response.

So what if I mask everything but the response w/ -100?

generate accurate

responses rather

than memorizing

instructions

$\star$ Masking Target Token IDs

Input

$[2, 106, 318, 281, 2084, \dots, 13]$

Target

$[100, 100, 100, 100, \dots, 13, 50256]$

K/V Bit
 Instruction training w/
 Loss over instructions

Token IDs corresponding to the input text

Instruction tokens

are replaced by -100

to generate accurate